Topic 1 · C Memory & Pointers: Worked Execution Guide

[!!!] CORE MUST-KNOW
MEMORY & POINTERS · 15 MARKS

Dynamic Allocation, String Boundaries & Base Address Invariant ·
CROSS-REF SPEC §1.2

WORKED EXAMPLE 1 String Duplication (malloc, strlen + 1, free Base-Address)

LETHAL EXAM TRAP

Past Exam Prompt: *"Given string `s = \"Algo\"`. Allocate exact heap memory, duplicate characters, and show why calling `free()` after pointer traversal causes undefined heap corruption."*

STEP 1 **Allocate Exact Memory: `strlen(s) + 1`**

`strlen(\"Algo\") = 4` (counts printable characters only!).
To store null terminator `\'\0\'`, allocation MUST be: `malloc(strlen(s) + 1) = 5 bytes`!
Omitting +1: Causes buffer overflow overwrite into adjacent heap chunks → Memory Corruption!

MEMORY MAP **5 Contiguous Heap Bytes (Base Address = 0x1000)**

char *base_ptr = 0x1000

0x1000	0x1001	0x1002	0x1003	0x1004
`'A'`	`'l'`	`'g'`	`'o'`	`'\0'`

STEP 2 **The Base-Address Invariant for `free(ptr)`**

Suppose code traverses the string with: `while (*p) p++;`. Now pointer `p = 0x1004`.
FATAL BUG: Calling `free(p)` passes an interior address! The C runtime heap manager only tracks the **allocation header** immediately preceding `0x1000`.
Rule: You MUST save and pass the original base address: `free(base_ptr); base_ptr = NULL;`

WORKED EXAMPLE 2 Linked List Head Modification (Double Pointer `node_t **`)

CRITICAL MECHANIC

Past Exam Prompt: *"Write a C function to delete the head of a singly linked list. Explain why `void deleteHead(node_t *head)` fails, and show the exact double pointer rewiring trace."*

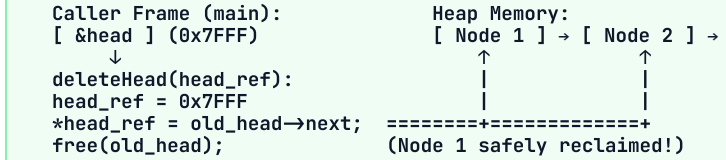
DIAGNOSIS **Why Single Pointer (`node_t *head`) Fails**

C parameters are strictly **pass-by-value**! The function receives a local copy of the head pointer. Reassigning `head = head→next;` only changes the local copy inside the function frame; the caller's head variable in `main()` remains completely unchanged!

SOLUTION **Double Pointer Rewiring Trace (`node_t **head_ref`)**

Pass the memory address of the caller's head variable: `deleteHead(&head);`
1. **Dereference once:** `node_t *old_head = *head_ref;` (points to 1st node).
2. **Rewire caller's pointer:** `*head_ref = old_head→next;` (caller's head now points to 2nd node!).
3. **Release memory:** `free(old_head);`. No memory leaks, caller successfully updated!

VECTOR MAP **Double Pointer Indirection Flow**



DANGLING POINTER TRAP
`free(p)` releases memory back to OS, but **variable p still holds the old address!** Dereferencing `*p` afterwards is Undefined Behavior. Always set `p = NULL;`!

SIZEOF STRUCT PADDING
Never hardcode struct sizes! Hardware word alignment inserts padding bytes. Always use `sizeof(struct student)` instead of summing field sizes.

DYNAMIC ARRAY REALLOCATION
When array is full, double capacity: `new_cap = cap * 2`. Geometric doubling achieves **amortized O(1) append time**! Linear increment (+10) degrades append to O(*N*).

C MEMORY LEAKS CHECKLIST
Every successful `malloc()` must have exactly one corresponding `free()`. For linked lists, free nodes sequentially from head to tail: save `next` before calling `free(curr)`!

Topic 2 · Complexity & Loops: Worked Analysis Guide

[!!!] CORE MUST-KNOW
COMPLEXITY ANALYSIS · 15 MARKS

Loop Summations, Geometric Stepping & Big-O Formal Bounds ·
CROSS-REF SPEC §2.2

WORKED EXAMPLE 1 Nested Loops with Geometric Step (i *= 2)

HIGH EXAM YIELD

Past Exam Code Snippet:

```
for (int i = 1; i ⩽ N; i *= 2) { for (int j = 0; j < N; j++) { sum += A[i][j]; } }
```

Exam Task: Formulate the exact summation, evaluate the closed-form bound, and state the tight Big-Θ complexity.

STEP 1 Trace Loop Iteration Frequencies

- **Outer Loop:** Value of `i` takes values `1, 2, 4, 8, ..., 2^k ≤ N`. Solving `2^k ≤ N` yields `k ≤ ⌊log&sub2; N⌋`. The outer loop executes exactly **⌊log&sub2; N⌋ + 1 times**.
- **Inner Loop:** For every iteration of `i`, `j` runs from 0 to `N-1`, doing exactly **N iterations**.

STEP 2 Formulate and Evaluate Closed-Form Sum

`Total Operations = ∑_{k=0}^{\lfloor\log_2 N floor} N = N × (⌊\log_2 N floor + 1) = N \log_2 N + N.`
Dropping lower-order terms and constant coefficients → **Tight Bound: Θ(N log N)**!

FORMAL BOUNDS Mathematical Definitions Reference

NOTATION	FORMAL MATHEMATICAL DEFINITION	EXAM INTUITION
Big-O: O(g(n))	∃ c > 0, *n*&sub0; > 0 s.t. 0 ≤ *f*(*n*) ≤ c · *g*(*n*), ∀ *n* ≥ *n*&sub0;	**Upper bound** (Worst-case ceiling)
Big-Ω: Ω(g(n))	∃ c > 0, *n*&sub0; > 0 s.t. 0 ≤ c · *g*(*n*) ≤ *f*(*n*), ∀ *n* ≥ *n*&sub0;	**Lower bound** (Best-case floor)
Big-Θ: Θ(g(n))	*f*(*n*) ∈ O(*g*(*n*)) AND *f*(*n*) ∈ Ω(*g*(*n*)) simultaneously	**Asymptotically tight** (Exact growth rate)

COMMON SUMMATION FORMULA

- Arithmetic: `∑_{i=1}^N i = N(N+1)/2 ∈ θ(N^2)`.
- Geometric: `∑_{i=0}^k 2^i = 2^{k+1} - 1 ∈ θ(2^k)`.
- Harmonic: `∑_{i=1}^N 1/i ∈ θ(log N)`.

GROWTH HIERARCHY

`1 < log log N < log N < √N < N < N log N < N^2 < N^3 < 2^N < N!`
Polynomial always beats logarithmic; exponential crushes polynomial!

WORKED EXAMPLE 2 Recurrence Relation Unrolling (Repeated Substitution)

MATHEMATICAL PROOF

Past Exam Recurrence: `T(N) = 2 T(N / 2) + cN`, with base case `T(1) = d`. (Divide-and-conquer pattern).
Exam Task: Solve using repeated substitution (unrolling). Show step 1, 2, general pattern at step k, base case, and final Big-Θ.

STEP 1 & 2 Substitute T(N/2) and T(N/4)

- **Step 1:** `T(N) = 2 [ 2 T(N/4) + c(N/2) ] + cN = 4 T(N/4) + 2cN`.
- **Step 2:** `T(N) = 4 [ 2 T(N/8) + c(N/4) ] + 2cN = 8 T(N/8) + 3cN`.

STEP 3 Identify General Invariant at Step k

`T(N) = 2^k T(N / 2^k) + k · cN`.
The base case is reached when `N / 2^k = 1 → 2^k = N → k = \log_2 N`.

STEP 4 Substitute Base Case and Conclude

Substitute `2^k = N` and `k = \log_2 N`:
`T(N) = N · T(1) + (\log_2 N) · cN = d · N + c · N \log_2 N`.
Leading term dominates → **Strict Linear-Logarithmic Bound: Θ(N log N)**!

MASTER THEOREM Standard Form: `T(N) = a T(N / b) + f(N)` with critical exponent `c_{crit} = \log_b a`

CASE	CONDITION COMPARISON	ASYMPTOTIC SOLUTION	CLASSIC EXAMPLE
Case 1: Leaves Dominate	`f(N) = O(N^{c_{crit} - \epsilon})`	`T(N) = θ(N^{\log_b a})`	Strassen Matrix: *T(N)=7T(N/2)+O(N²)* → *Θ(N^{2.81})*
Case 2: Balanced Work	`f(N) = θ(N^{c_{crit}} \log^k N)`	`T(N) = θ(N^{\log_b a} \log^{k+1} N)`	**Mergesort:** *T(N)=2T(N/2)+cN* → *Θ(N log N)*
Case 3: Root Dominates	`f(N) = Ω(N^{c_{crit} + \epsilon})` + regularity	`T(N) = θ(f(N))`	*T(N)=2T(N/2)+N²* → *Θ(N²)*

MASTER THEOREM PITFALL

Cannot apply Master Theorem if `a` is not constant (e.g. `T(N) = 2^N T(N/2)`) or if `f(N)` is not polynomial (e.g. `T(N) = 2T(N/2) + N / \log N`). Use unrolling!

BINARY SEARCH RECURRENCE

`T(N) = T(N/2) + O(1)` → `a = 1, b = 2, c_{crit} = \log_2 1 = 0`.
Case 2 gives **Θ(log N)**!

Topic 3 · AVL Trees: Worked Balance & Rotation Guide

[!!!] CORE MUST-KNOW
AVL ROTATIONS · 15–20 MARKS

Step-by-step Balance Factors & Pointer Rewiring · CROSS-REF SPEC §3.3

WORKED EXAMPLE 1 AVL Balance Factor & Single Right (LL) Rotation

CRITICAL MECHANIC

Past Exam Prompt: *"Given a valid AVL tree with keys `[30, 20]`. Key 10 is inserted. Compute Balance Factor BF = H(left) - H(right) for every ancestor, identify the imbalance type, and draw the rotated tree."*

AVL Invariant: For every node, balance factor `BF ∈ {-1, 0, +1}`. If `|BF| ≥ 2`, the tree is INVALID and requires an immediate rotation at the **lowest unbalanced ancestor**!

STEP 1 BST Insert Key 10 & Compute Balance Factors

- Node 10 inserted as left child of 20. Height = 0, `BF(10) = 0`.
- Parent 20: Left height = 0, right = -1 → `BF(20) = 0 - (-1) = +1` (OK).
- Root 30: Left child height = 1, right = -1 → `BF(30) = 1 - (-1) = +2 ≥ 2 (IMBALANCE!)`

STEP 2 Classify Imbalance: LL Case (Left-Left)

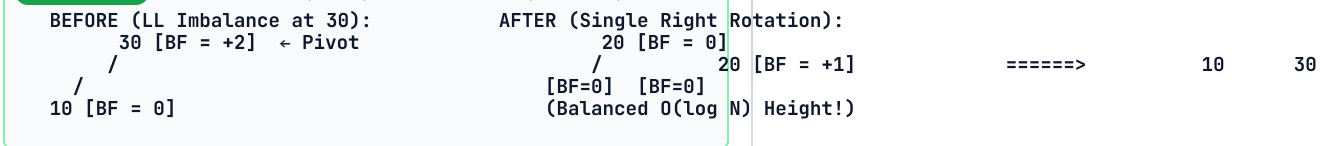
Imbalance discovered at node **30**. Cause: new key was inserted into the **Left** subtree of its **Left** child (20 → 10).
Prescription: Exactly **ONE Single Right Rotation** around node 30!

STEP 3 Execute Single Right Rotation (Clockwise)

1. Node **20** rises to become the new root of this subtree.
2. Node **30** is pushed down to become the **right child** of 20.
3. Any right subtree of 20 (if present) is handed over to become the left subtree of 30.

Result: Root = **20**, Left Child = **10**, Right Child = **30**. All BFs are now **0**!

VECTOR MAP Before Rotation (Chain) vs After Rotation (Balanced)



WORKED EXAMPLE 2 LR Double Rotation Step-by-Step (The "Zig-Zag" Case)

MOST TESTED TRAP

Past Exam Prompt: *"Given tree with keys `[30, 10]`. Key 20 is inserted. Show why a single rotation fails, trace the two constituent rotations, and write the final tree structure."*

DIAGNOSIS Why Single Rotation FAILS on Zig-Zag (LR)

Node 20 is inserted as the **right child of 10**. Root 30 has `BF = +2`, but child 10 has `BF = -1` (opposite signs!).
A single right rotation around 30 produces an equally crooked tree. A **Double Rotation** is mathematically mandatory!

PHASE 1 Left Rotation on Child Node 10 (Straightens Zig-Zag into LL)

Rotate child node **10 counter-clockwise**:
- Node **20** rises to become the left child of 30.
- Node **10** becomes the left child of 20.

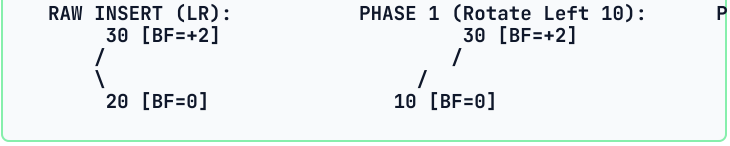
State: The tree is now a straight **LL configuration**: `30 → 20 → 10`!

PHASE 2 Right Rotation on Root Node 30 (Restores Balance)

Rotate root node **30 clockwise** around 20:
- Node **20** ascends to become the global root!
- Node **30** descends to become the right child of 20.
- Node **10** remains the left child of 20.

Final Valid AVL: Root = **20**, Left = **10**, Right = **30**. Perfectly balanced!

VECTOR MAP Phase 1 & Phase 2 Double Rotation Flow



ROTATION DECISION MATRIX

- **BF(Root) = +2, BF(Left) = +1 → LL**: Single Right.
- **BF(Root) = -2, BF(Right) = -1 → RR**: Single Left.
- **BF(Root) = +2, BF(Left) = -1 → LR**: Double (Left then Right).
- **BF(Root) = -2, BF(Right) = +1 → RL**: Double (Right then Left).

HEIGHT VS DEPTH FORMULA

Empty tree height = -1, Leaf height = 0.
`Height(u) = 1 + max(Height(left), Height(right))`.
Always recalculate heights from bottom up!

EXAMINER 2-CHILD DELETION TRICK

To delete node with 2 children: **swap with In-order Successor** (smallest key in right subtree) OR **In-order Predecessor** (largest in left). Then delete the leaf!

TIME COMPLEXITY SUMMARY

- **Search:** $O(\log N)$ worst-case.
- **Insert:** $O(\log N)$ search + at most **1 rotation** (single or double) → $O(\log N)$.
- **Delete:** $O(\log N)$ search + up to **$O(\log N)$ rotations** up to root!

Topic 4 · The Binary Heap: Worked Trace Guide

[!!!] CORE MUST-KNOW
WORKED TRACES · 15–20 MARKS

Concrete Worked Examples with Numbers
· CROSS-REF SPEC §4.2.4

WORKED EXAMPLE 1 Up-Heap (Insert 15 into Max-Heap of size N = 6)

HIGH EXAM YIELD

Past Exam Prompt: *"Given a 1-based max-heap array `A = [16, 14, 10, 8, 7, 9]`. Trace inserting **15**. Show every comparison, pointer index, and the final array state."*

STATE 0 **Initial Array (N = 6, Valid Max-Heap)** Root at index 1

i = 1	i = 2	i = 3	i = 4	i = 5	i = 6	i = 7
16	14	10	8	7	9	∅

STEP 1 **Append Key at Frontier `A[N+1]` → `N = 7`**

Put **15** at next open slot `A[7]`. Parent is at index `⌊7 / 2⌋ = 3` (holding value **10**).
Comparison 1: Compare `A[7]` (15) with parent `A[3]` (10). Since **15 > 10**, heap order is violated → **SWAP `A[7]` and `A[3]`!**

STATE 1 **After First Swap (15 moved to index 3, 10 pushed to index 7)**

i = 1	i = 2	i = 3	i = 4	i = 5	i = 6	i = 7
16	14	15	8	7	9	10

STEP 2 **Sift Up & Verify Root Invariant**

Current index of 15 is `i = 3`. Parent is at index `⌊3 / 2⌋ = 1` (holding value **16**).
Comparison 2: Compare `A[3]` (15) with parent `A[1]` (16). Since **15 ≤ 16**, heap order is satisfied → **STOP!**

FINAL **Resulting Array: `[16, 14, 15, 8, 7, 9, 10]` • Total Comparisons = 2**

i = 1	i = 2	i = 3	i = 4	i = 5	i = 6	i = 7
16	14	15	8	7	9	10

WORKED EXAMPLE 2 Floyd's Bottom-Up `makeHeap` (N = 7, Θ(N) Trace)

MATHEMATICAL PROOF

Past Exam Prompt: *"Given raw unsorted array `A = [4, 10, 3, 5, 1, 9, 2]`. Trace Floyd's bottom-up build algorithm. Show which node is downheaped first, every pass, and the final valid max-heap."*

Execution Axiom: Leaves require 0 downheaps! Loop begins strictly at **last internal node**: `i_start = ⌊7 / 2⌋ = 3`. Order: `downheap(3)` → `downheap(2)` → `downheap(1)`.

STATE 0 **Raw Unsorted Array (Before Heapify)** Internal nodes: 1, 2, 3

i = 1	i = 2	i = 3	i = 4	i = 5	i = 6	i = 7
4	10	3	5	1	9	2

PASS 1 **`i = 3`: Subtree rooted at `A[3] = 3` (Children: `A[6] = 9`, `A[7] = 2`)**

1. **Compare children:** 9 vs 2 → larger child is **9** at index 6.
2. **Compare with parent:** 9 > 3 → **SWAP `A[3]` and `A[6]`!** Array: `[4, 10, 9, 5, 1, 3, 2]`. (2 comparisons)

PASS 2 **`i = 2`: Subtree rooted at `A[2] = 10` (Children: `A[4] = 5`, `A[5] = 1`)**

1. **Compare children:** 5 vs 1 → larger child is **5** at index 4.
2. **Compare with parent:** 10 ≥ 5 → **No violation! No swap!** Array stays: `[4, 10, 9, 5, 1, 3, 2]`. (2 comparisons)

PASS 3 **`i = 1`: Subtree rooted at `A[1] = 4` (Two-level Domino Downheap!)**

• **Level 1:** Compare children 10 vs 9 → larger 10. Compare 10 > 4 → **SWAP `A[1]` & `A[2]`!**
• **Level 2 (Follow 4 to index 2):** Children are 5 and 1. Larger is 5. Compare 5 > 4 → **SWAP `A[2]` & `A[4]`!**

FINAL **Built Max-Heap: `[10, 5, 9, 4, 1, 3, 2]` • Root is Global Max (10)**

i = 1	i = 2	i = 3	i = 4	i = 5	i = 6	i = 7
10	5	9	4	1	3	2

UP-HEAP ASYMMETRY RULE
Comparisons per level = exactly 1.
Node only checks its single parent.
Worst-case comparisons ≤ ⌊log&sub2; *N*⌋ = 2 comparisons for *N* = 7!

CHEATSHEET RAPID ANCHOR
• **Leaf Indices:** ⌊7/2⌋ + 1 = 4 → Indices 4, 5, 6, 7 are leaves.
• **Formulas Table:** See `COMP20003_Full_Exam_Spec.pdf` §4.2.2.

DOWN-HEAP ASYMMETRY RULE
Comparisons per level = exactly 2.
• *Comp 1:* Compare left vs right child to pick the winner.
• *Comp 2:* Compare the winner against the parent node.

LETHAL 0-MARKS TRAP
Pitfall: Writing Floyd takes O(*N* log *N*) because it calls downheap *N*/2 times.
Full Mark: Height is low for most nodes!
$\sum (h / 2^h)$ converges to 2 → **strict Θ(*N*) linear time**.

Topic 5 · Hash Tables: Worked Collision & Probe Guide

[!!!] CORE MUST-KNOW
PROBE TRACES · 10–15 MARKS

Exact Probe Counts & Clustering Avoidance
· CROSS-REF SPEC §5.2

WORKED EXAMPLE 1 Linear Probing Insertion & Probe Counting (M = 7)

PROBE COUNT FORMULA

Past Exam Prompt: *"Given hash table size M = 7 with hash function h(k) = k mod 7. Insert keys `[10, 17, 24, 8]` using Linear Probing. Record each probe count and show final table state."*

Probe Formula: `h_i(k) = (h(k) + i) mod M` for `i = 0, 1, 2, ...`
CRITICAL EXAM RULE: A "probe" is counted for **EVERY slot inspected**, including the home slot and the final empty slot where insertion succeeds!

KEY 10 `h(10) = 10 mod 7 = 3`
Slot 3 is empty. Insert 10 at `T[3]`. **Probes = 1** (inspected slot 3).

KEY 17 `h(17) = 17 mod 7 = 3` **(Collision!)**
- Probe 1: Slot 3 occupied by 10 → collision!
- Probe 2: Try `(3 + 1) mod 7 = 4`. Slot 4 is empty. Insert 17 at `T[4]`. **Probes = 2**.

KEY 24 `h(24) = 24 mod 7 = 3` **(Double Collision!)**
- Probe 1: Slot 3 occupied (10).
- Probe 2: Slot 4 occupied (17).
- Probe 3: Try `(3 + 2) mod 7 = 5`. Slot 5 is empty. Insert 24 at `T[5]`. **Probes = 3**.

KEY 8 `h(8) = 8 mod 7 = 1`
Slot 1 is empty. Insert 8 at `T[1]`. **Probes = 1**.

FINAL TABLE Primary Cluster formed at slots 3, 4, 5 • Total Probes = 1 + 2 + 3 + 1 = 7

0	1	2	3	4	5	6
∅	8	∅	10	17	24	∅

PRIMARY CLUSTERING FLAW
Linear probing causes adjacent occupied slots to merge into long runs. Any key that lands anywhere in the cluster must traverse the whole cluster!

LOAD FACTOR A LIMITS
`α = N / M`. In Linear Probing, α MUST be ≤ 1.0 (performance degrades drastically when α > 0.7). In Separate Chaining, α can exceed 1.0!

WORKED EXAMPLE 2 Double Hashing Step-by-Step (Eliminating Clustering)

EXAM MANDATORY RULES

Past Exam Prompt: *"Given M = 7, h1(k) = k mod 7, and secondary hash h2(k) = 5 - (k mod 5). Insert keys `[10, 17]` using Double Hashing. Show how collision resolution jumps differ from linear probing."*

THE TWO MANDATORY RULES FOR h2(k):
1. `h2(k) ≠ 0` **for all keys** (otherwise step size is 0 → infinite loop on same slot!).
2. `gcd(h2(k), M) == 1` (step size and table size must be coprime so ALL M slots are reachable).

KEY 10 `h1(10) = 10 mod 7 = 3`
Slot 3 is empty. Insert 10 at `T[3]`. **Probes = 1**.

KEY 17 `h1(17) = 3` **(Collision!) → Calculate Custom Step** `h2(17)`
- Probe 1: Slot 3 occupied by 10.
- Step calculation: `h2(17) = 5 - (17 mod 5) = 5 - 2 = 3`.
- Probe 2: Try `(h1 + 1 × h2) mod M = (3 + 3) mod 7 = 6`.
- Slot 6 is empty! Insert 17 at `T[6]`. **Probes = 2**.

SCATTERED RESULT Keys dispersed across table without forming consecutive clusters!

0	1	2	3	4	5	6
∅	∅	∅	10	∅	∅	17

SEARCH DELETION TOMBSTONE TRAP
In open addressing, you CANNOT simply set a slot to empty when deleting! A vacant slot terminates future searches prematurely. You MUST mark it as **TOMBSTONE (Deleted)**!

AVERAGE SEARCH COMPLEXITY
- **Separate Chaining:** Successful search ≈ `1 + α/2`.
- **Linear Probing:** Successful ≈ `0.5 × (1 + 1/(1-α))`.

Always choose **M = Prime** to break patterns!

Topic 6 · Quicksort: Hoare Partition Worked Guide

[!!!] CORE MUST-KNOW
HOARE TRACE · 15–20 MARKS

Two-Pointer Inversion Swaps & Pivot Invariant · CROSS-REF SPEC §6.3

WORKED EXAMPLE 1 Hoare Two-Pointer Partition Step-by-Step

EXAM TRACE PROTOCOL

Past Exam Prompt: *"Given subarray `A = [5, 3, 8, 4, 2, 7, 1, 9]`. Pivot is leftmost item p = 5. Trace Hoare's two-pointer partition. Show every pointer movement of L and R, all element swaps, and final pivot position."*

STATE 0 **Initial Array (Pivot p = A[0] = 5, L = 1, R = 7)**

0 (P)	1 (L)	2	3	4	5	6	7 (R)
5	3	8	4	2	7	1	9

STEP 1 **Advance Pointers to Find Inversion**

- Advance **L right** while `A[L] < p`: Skips 3 (<5). Stops on **8** at `L = 2` (8 ≥ 5).
- Advance **R left** while `A[R] > p`: Skips 9 (>5), skips 7 (>5). Stops on **1** at `R = 6` (1 ≤ 5).

Since `L < R` (2 < 6), an inversion exists → **SWAP `A[2]` and `A[6]`!**

STATE 1 **After First Swap (8 and 1 swapped)**

0 (P)	1	2	3	4	5	6	7
5	3	1	4	2	7	8	9

STEP 2 **Advance Pointers until Crossover**

- Advance L: Skips 4 (<5), skips 2 (<5). Stops on **7** at `L = 5`.
- Advance R: Stops on **2** at `R = 4`.

Pointers have crossed (L > R, 5 > 4)! STOP scanning immediately!

STEP 3 **Swap Pivot with R**

Swap pivot `A[0]` (5) with boundary element `A[R]` (`A[4] = 2`)!

Final Partitioned Array: `[2, 3, 1, 4, 5, 7, 8, 9]`. Pivot 5 is in its permanent sorted place!

MASTER MATRIX Complete 7-Algorithm Comparison

HIGH EXAM YIELD

ALGORITHM	BEST TIME	AVG TIME	WORST TIME	AUX SPACE	STABLE?	IN-PLACE?	ADAPTIVE?
Selection Sort	$\Theta(N^2)$	$\Theta(N^2)$	$\Theta(N^2)$	$O(1)$	No	Yes	No
Insertion Sort	**$\Theta(N)$**	**$\Theta(N^2)$**	**$\Theta(N^2)$**	**$O(1)$**	**Yes**	**Yes**	**Yes (sorted)**
Quicksort	$\Theta(N \log N)$	$\Theta(N \log N)$	$\Theta(N^2)$	$O(\log N)$	No	Yes	No
Mergesort (Array)	$\Theta(N \log N)$	$\Theta(N \log N)$	$\Theta(N \log N)$	**$O(N)$**	**Yes**	No	No
Mergesort (List)	**$\Theta(N \log N)$**	**$\Theta(N \log N)$**	**$\Theta(N \log N)$**	**$O(1)$**	**Yes**	**Yes**	**Natural: Yes**
Heapsort	$\Theta(N \log N)$	$\Theta(N \log N)$	$\Theta(N \log N)$	$O(1)$	No	Yes	No

WORKED EXAMPLE 2 Two-Pointer Merge Step-by-Step

MERGE TRACE

Merge two sorted halves `L = [2, 5, 8]` and `R = [3, 4, 9]` into auxiliary array `B`.

STEP 1 **Compare `L[0] = 2` vs `R[0] = 3`**

2 ≤ 3 → Copy 2 to `B[0]`. Increment L pointer. `B = [2]`.

STEP 2 **Compare `L[1] = 5` vs `R[0] = 3`**

3 < 5 → Copy 3 to `B[1]`. Increment R pointer. `B = [2, 3]`.

STEP 3 **Compare `L[1] = 5` vs `R[1] = 4`**

4 < 5 → Copy 4 to `B[2]`. Increment R pointer. `B = [2, 3, 4]`.

STEP 4 **Flush Remaining Elements**

Compare 5 vs 9 → Copy 5, then 8, then leftover 9. **Final:** `[2, 3, 4, 5, 8, 9]`. Exactly **$\Theta(N)$ time**!

PIVOT CHOICE & WORST CASE

Fixed end pivot on pre-sorted input causes `0 / (N-1)` splits → **$\Theta(N^2)$ worst case!**

Fix: Median-of-Three (first, mid, last) guarantees expected $O(N \log N)$.

HOARE VS LOMUTO TRAP

Hoare partition takes on average **3× fewer swaps** than Lomuto single-pointer partition. Lomuto degrades severely on duplicate keys!

WHY HEAPSORT IS UNSTABLE

Swapping the root with the last leaf element `A[N]` jumps equal elements over large spans of memory, completely destroying original relative order!

CUTOFF TO INSERTION SORT

Stopping Quicksort recursion when subarray size ≤ 10 and finishing with Insertion Sort boosts CPU cache speed, but **DOES NOT change asymptotic Big-O**!

Topic 7 · Dijkstra: Worked State Matrix Trace Guide

[!!!] CORE MUST-KNOW
DIJKSTRA TRACE · 15–20 MARKS

Step-by-Step Priority Queue Relaxation & Path Recovery · CROSS-REF SPEC §7.3

WORKED EXAMPLE 1 Dijkstra Execution Log Matrix (Vertices A, B, C, D)

EXAM RELAXATION TABLE

Past Exam Graph: 4 vertices with directed edges: `(A→B: 2)`, `(A→C: 5)`, `(B→C: 1)`, `(B→D: 6)`, `(C→D: 2)`. Start vertex: **A**.
Exam Task: Fill the strict execution trace table. Record which vertex is finalized at each step and all distance relaxations.

Edge Relaxation Axiom: If `dist[u] + weight(u, v) < dist[v]`, update: `dist[v] = dist[u] + weight(u, v)` and set `prev[v] = u`!

ITERATION	POP MIN (U)	RELAX EDGES (U → V)	DIST[A]	DIST[B]	DIST[C]	DIST[D]	FINALIZED SET S
Init	—	Initialize start vertex A	0	∞	∞	∞	∅
Step 1	A (dist=0)	Relax (A,B)=2 → B: 2 Relax (A,C)=5 → C: 5	0*	2 (A)	5 (A)	∞	{A}
Step 2	B (dist=2)	Relax (B,C)=2+1=3 < 5 → C: 3 Relax (B,D)=2+6=8 → D: 8	0*	2*	3 (B)	8 (B)	{A, B}
Step 3	C (dist=3)	Relax (C,D)=3+2=5 < 8 → D: 5	0*	2*	3*	5 (C)	{A, B, C}
Step 4	D (dist=5)	No outgoing edges	0*	2*	3*	5*	{A, B, C, D}

PATH RECOVERY **Backtrack `prev[]` pointers from destination D:** `D ← (prev[D]=C) ← (prev[C]=B) ← (prev[B]=A)` → Shortest Path: A → B → C → D (Total Cost = 5).

WORKED EXAMPLE 2 BFS vs DFS Queue/Stack Trace Protocol

CORE TRAVERSAL

PROPERTY	BREADTH-FIRST SEARCH (BFS)	DEPTH-FIRST SEARCH (DFS)
Underlying Data Structure	Queue (FIFO)	Stack (LIFO) or Function Call Stack
Exploration Pattern	Concentric ripples (level by level)	Deep plunge down a path until dead end
Shortest Path Guarantee	Guarantees minimum hops on unweighted graphs!	NO guarantee of shortest path
Memory Space Bound	O(branching factor ^ depth) (Can be huge)	O(maximum path depth) (Memory frugal)
Signature Exam Application	Shortest path in unweighted mazes / networks	Topological Sort on DAGs, Cycle Detection

MINIMUM SPANNING TREE Prim vs Kruskal Battle

DESIGN CHOICE

DIMENSION	PRIM'S ALGORITHM	KRUSKAL'S ALGORITHM
Strategy	**Vertex-based:** Grows a single tree outwards	**Edge-based:** Sorts all edges, joins forests
Helper Data Structure	Priority Queue (Min-Heap)	**Disjoint Set (Union-Find)** with path compression
Time Complexity	$O(\|E\| \log \|V\|)$	$O(\|E\| \log \|E\|) = O(\|E\| \log \|V\|)$
Best Suited For	**Dense graphs** ($\|E\| \approx \|V\|^2$)	**Sparse graphs** ($\|E\| \approx \|V\|$)

NEGATIVE EDGE FATAL FLAW
Dijkstra assumes once a vertex is finalized in **S**, its distance can NEVER decrease. A negative edge breaks this greedy invariant! Use **Bellman-Ford** for negative edges.

TIME COMPLEXITY BOUNDS
- **Binary Heap PQ:** $O((|V| + |E|) \log |V|)$.
- **Dense Array PQ:** $O(|V|^2)$ → optimal for dense graphs ($|E| \approx |V|^2$)!

GRAPH REPRESENTATION BOUNDS
- **Adjacency Matrix:** Space $\Theta(|V|^2)$. Edge check $O(1)$. Iterate neighbours $\Theta(|V|)$.
- **Adjacency List:** Space $\Theta(|V| + |E|)$. Iterate neighbours $\Theta(\deg(u))$. Optimal for sparse!

TOPOLOGICAL SORT RULE
Valid ONLY on **Directed Acyclic Graphs (DAGs)**! Reverse of DFS post-order finish times gives valid topological order.